

KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY, KHULNA -9203

PROJECT NAME: Design and Simulation of ROM, PLA and PAL Using Gate-Level and NMOS-Level Logic in EDA Playground

PROJECT COURSE NO: ECE 4102

COURSE NAME: VLSI Design and Technology Laboratory

Group: B

Submitted To:	**Submitted by:**
Dr. Sheikh Md. Rabiul Islam Professor, Department of Electronics and Communication Engineering, KUET **Naymur Rahman** Assistant Professor, Department of Electronics and Communication Engineering, KUET	**Md. Akib Jawad** **Roll - 2109059**

1. Objectives

The main objectives of this project are:

- To design a ROM using gate-level Verilog.
- To design a ROM using NMOS transistor-level logic.
- To design a PLA using gate-level logic.
- To design a PLA using NMOS transistor-level logic.
- To design a PAL using gate-level logic.
- To design a PAL using NMOS transistor-level logic.
- To implement Boolean functions using minterms and Sum-of-Products (SOP) representation.
- To understand the operation of NOT, AND, and OR gates at the structural (gate primitive) level.
- To understand pseudo-NMOS level implementation using pull-up loads and NMOS pull-down networks.
- To verify all possible 4-bit input combinations through simulation.
- To generate waveform data using a VCD file and observe the circuit operation in EPWave.
- To compare the implementation concepts of ROM, PLA, and PAL.

2. Introduction

Programmable Logic Devices (PLDs) are digital circuits that can be programmed to implement different Boolean functions without designing a separate hardware circuit for each function. They provide a flexible and efficient approach for implementing digital logic systems. Some commonly used programmable logic structures include Read-Only Memory (ROM), Programmable Logic Array (PLA), and Programmable Array Logic (PAL).

In this project, a four-input Boolean logic system is designed using the input variables A, B, C, and D, where A is the most significant bit (MSB) and D is the least significant bit (LSB). The system produces two output functions, Y1 and Y2, which are generated according to an assigned roll-number-based rule.

- The output function Y1 is derived from the Sum of Products (SOP) minterm representation of the student's own roll number. Each distinct digit in the roll number is considered as a required minterm index, where each digit (0-9) represents a valid 4-bit minterm value (0-15).
- The output function Y2 is obtained in the same way from the SOP (minterm) form of the immediately preceding roll number.

For this project, the roll number used for generating Y1 is 2109059, and the preceding roll number used for generating Y2 is 2109058. The distinct digits appearing in each roll number are taken as the required minterms for the corresponding Boolean function.

Derivation of Y1 (roll 2109059):

Digits of 2109059 -> 2, 1, 0, 9, 0, 5, 9. The distinct digits, in ascending order, are 0, 1, 2, 5, 9.

$$Y1 = \Sigma m(0, 1, 2, 5, 9)$$

Derivation of Y2 (preceding roll 2109058):

Digits of 2109058 -> 2, 1, 0, 9, 0, 5, 8. The distinct digits, in ascending order, are 0, 1, 2, 5, 8, 9.

$$Y2 = \Sigma m(0, 1, 2, 5, 8, 9)$$

The minterms 0, 1, 2, 5 and 9 are common to both outputs, while minterm 8 appears only in Y2. This single extra minterm (input 1000) is the key difference between the two output functions and is what distinguishes the current roll (2109059) from the preceding roll (2109058).

After K-map simplification, the reduced Boolean expressions used throughout the project are:

$$Y1 = A'C'D + B'C'D + A'B'D'$$

$$Y2 = B'C' + A'C'D + A'B'D'$$

The terms A'C'D and A'B'D' are common to both outputs. This sharing of product terms is exploited in the PLA and PAL designs, where the shared terms are generated once and reused, while B'C'D (for Y1) and B'C' (for Y2) are the terms unique to each output.

The same Boolean functions are implemented in different hardware structures (ROM, PLA, PAL, each at gate level and at pseudo-NMOS transistor level) so that the differences between them can be understood practically.

3. Basic Theory

3.1 Boolean Logic

Boolean logic is the fundamental concept used in digital electronics and forms the basis for designing and analyzing logic circuits. It operates using two logical states: 0 (LOW) and 1 (HIGH). The main Boolean operations used in this project are described below.

NOT Operation

The NOT operation, also known as an inverter operation, produces the complement of its input: Y = A'. If the input A is 0, the output Y becomes 1, and if A is 1, the output Y becomes 0. In this project, the pseudo-NMOS level implementation of the NOT gate is realized using an NMOS transistor with a pull-up load.

3.2 AND Operation

The AND operation produces an output of 1 only when all its input variables are equal to 1. For example, the product term A'C'D becomes 1 only for the input combination A=0, C=0, D=1, which corresponds to input combinations such as 0001 (minterm 1) or 0101 (minterm 5) depending on B. In the gate-level implementation, the AND operation is represented using the AND primitive.

3.3 OR Operation

The OR operation produces an output of 1 when at least one of its input terms is equal to 1. It is represented by the "+" operator. For example, Y1 = A'C'D + B'C'D + A'B'D' combines three product terms using OR; if any one of the three terms becomes 1, the output Y1 also becomes 1.

4. Minterm Representation

A minterm is a product term that represents exactly one combination of the input variables. For four variables there are 2^4 = 16 possible combinations, ranging from m0 to m15. The distinct minterms relevant to this project are listed below.

Minterm	ABCD	Boolean expression
m0	0000	A'B'C'D'
m1	0001	A'B'C'D
m2	0010	A'B'CD'
m5	0101	A'BC'D
m9	1001	AB'C'D

In addition, minterm m8 (ABCD = 1000, expression AB'C'D') is required only for Y2, since digit 8 appears only in the preceding roll number 2109058.

5. Canonical SOP and K-map Simplification

SOP means Sum of Products. In canonical SOP form, every minterm is written out as a full product term of all four variables, and these terms are combined through OR.

For this project:

$$Y1 = \Sigma m(0, 1, 2, 5, 9)$$

In canonical (unsimplified) minterm form:

$$Y1 = A'B'C'D' + A'B'C'D + A'B'CD' + A'BC'D + AB'C'D$$

$$Y2 = \Sigma m(0, 1, 2, 5, 8, 9)$$

$$Y2 = A'B'C'D' + A'B'C'D + A'B'CD' + A'BC'D + AB'C'D' + AB'C'D$$

Using a 4-variable Karnaugh map (with A, B as row variables and C, D as column variables), the minterms of Y1 are grouped as follows: m1 and m5 combine to give A'C'D; m1 and m9 combine to give B'C'D; and m0 and m2 combine to give A'B'D'. This gives the simplified expression:

$$Y1 = A'C'D + B'C'D + A'B'D'$$

For Y2, the K-map grouping is: m0, m1, m8 and m9 combine into a single quad giving B'C'; m1 and m5 combine to give A'C'D; and m0 and m2 combine to give A'B'D'. This gives the simplified expression:

$$Y2 = B'C' + A'C'D + A'B'D'$$

This simplified SOP representation is important because programmable logic devices such as PLA and PAL are naturally suited to implementing AND-OR logic, and it is these reduced expressions — not the canonical forms — that are directly implemented in the Verilog designs of this project.

6. ROM Implementation

6.1 ROM Theory

ROM stands for Read-Only Memory. A ROM can be used to implement a Boolean function by storing the required output value for every possible input combination, effectively acting as a lookup table addressed by the input variables.

For this project, as specified in the project structure, the ROM receives four inputs A, B, C, D and produces only a single output, Y1.

The required minterms are m0, m1, m2, m5 and m9, so:

$$Y1 = \Sigma m(0, 1, 2, 5, 9) = A'C'D + B'C'D + A'B'D'$$

7. ROM NMOS-Level Design

At the transistor level, digital logic in this project is constructed using pseudo-NMOS logic, where each output node is held HIGH by a resistive pull-up (modelled with the pullup() primitive) and pulled LOW only when the NMOS pull-down network conducts.

In pseudo-NMOS logic, NMOS transistors placed in series realize the AND of their (active-low) control signals, while transistors placed in parallel realize the OR of their control signals, and every stage is naturally inverting (it produces the complement of the AND/OR of its inputs).

The ROM NMOS design first generates the complemented inputs A', B', C', D' using pseudo-NMOS inverters. It then builds each required product term (A'C'D, B'C'D, A'B'D') as a pseudo-NMOS AND stage (series NMOS network, which produces the complement of the term), combines the three complemented product terms in a pseudo-NMOS OR stage (parallel NMOS network), and finally passes the result through one more pseudo-NMOS inverter to restore the correct, non-inverted polarity:

$$Y1 = A'C'D + B'C'D + A'B'D'$$

The ROM NMOS design therefore represents exactly the same Boolean function as the gate-level ROM, but its internal implementation is described using transistor-level pseudo-NMOS structures instead of primitive logic gates.

8. PLA Implementation

8.1 PLA Theory

PLA stands for Programmable Logic Array. A PLA consists of two programmable sections: a programmable AND plane, which generates the required product terms, and a programmable OR plane, which combines those product terms to generate the outputs.

8.2 PLA Output Equations

The AND plane generates four product terms: P1 = A'C'D, P2 = A'B'D', P3 = B'C'D and P4 = B'C'. The OR plane then connects these terms to the two outputs as follows:

$$Y1 = P1 + P2 + P3 \;=\; A'C'D + A'B'D' + B'C'D$$

$$Y2 = P1 + P2 + P4 \;=\; A'C'D + A'B'D' + B'C'$$

An important advantage of the PLA is that common product terms can be shared between multiple outputs. Here, P1 (A'C'D) and P2 (A'B'D') are used by both Y1 and Y2, while P3 (B'C'D) is required only for Y1 and P4 (B'C') only for Y2. This demonstrates the basic idea of a PLA: generate useful product terms once in the AND plane and connect them to whichever outputs need them through the programmable OR plane.

9. PLA NMOS-Level Design

The PLA pseudo-NMOS design uses the same transistor-level building blocks as the ROM: pseudo-NMOS inverters for the complemented inputs, pseudo-NMOS AND stages (series NMOS networks) for each product term, and pseudo-NMOS OR stages (parallel NMOS networks) for each output, followed by an output inverter to restore correct polarity.

The design first generates the complemented inputs nA, nB, nC, nD using pseudo-NMOS inverters, then generates the shared product terms P1 (A'C'D), P2 (A'B'D'), and the output-specific terms P3 (B'C'D) and P4 (B'C'). Terms P1 and P2 are wired into both output OR planes without being regenerated, directly reflecting the hardware sharing described in Section 8:

$$Y1 = P1 + P2 + P3$$

$$Y2 = P1 + P2 + P4$$

Therefore, the pseudo-NMOS PLA performs exactly the same logical operation as the gate-level PLA, but represents the AND plane and OR plane using transistor-level pull-up/pull-down structures instead of primitive AND/OR gates.

10. PAL Implementation

10.1 PAL Theory

PAL stands for Programmable Array Logic. A PAL is similar to a PLA because it uses product terms generated by an AND section and then combines them using an OR section. The important conceptual difference is:

- PLA: programmable AND plane + programmable OR plane.
- PAL: programmable AND plane + fixed OR plane.

For this project, the same required product terms are generated, but each output is given its own independent set of AND-plane terms rather than sharing wires directly, since the OR plane in a PAL is fixed rather than programmable:

$$Y1 = A'C'D + B'C'D + A'B'D' \quad (terms\ P1, P2, P3)$$

$$Y2 = B'C' + A'C'D + A'B'D' \quad (terms\ Q1, Q2, Q3)$$

The supplied PAL gate-level code implements these equations directly using individual product terms P1...P3 (for Y1) and Q1...Q3 (for Y2), each connected through its own fixed OR gate. Note that the terms A'C'D and A'B'D' are logically identical between the two outputs, but in the PAL structure they are generated twice — once in each output's own AND-plane row — because the OR plane cannot be rewired to share a single copy the way a PLA can.

11. PAL NMOS-Level Design

The PAL pseudo-NMOS design uses the same pseudo-NMOS building blocks as the pseudo-NMOS PLA. First, the input complements A', B', C', D' are generated using pseudo-NMOS inverters. Then the required product terms are generated independently for each output using pseudo-NMOS AND (series NMOS) structures, and the outputs are formed using fixed pseudo-NMOS OR (parallel NMOS) combinations followed by output inverters:

$$Y1 = P1 + P2 + P3 \quad (P1=A'C'D,\ P2=B'C'D,\ P3=A'B'D')$$

$$Y2 = Q1 + Q2 + Q3 \quad (Q1=B'C',\ Q2=A'C'D,\ Q3=A'B'D')$$

Thus, the PAL pseudo-NMOS implementation produces the same logical results as the PAL gate-level implementation, with the OR plane treated as fixed rather than programmable, and with each output's AND-plane transistors kept physically separate.

12. NMOS Transistor-Level Theory

In this project, the transistor-level designs use pseudo-NMOS logic, in which each internal node and output is modelled with a resistive pullup() load acting as the pull-up network, and an NMOS pull-down network (PDN) that conducts a path to GND when the output must be logic 0.

Pull-Down Network (PDN)

The NMOS network connects the output toward GND when the output should be logic 0. NMOS devices placed in series implement the AND of their (uninverted, active-high) control signals for the pull-down condition, so the resulting node is the complement of that AND — i.e. a NAND-like function. NMOS devices placed in parallel implement the OR of their control signals for the pull-down condition, giving a NOR-like function at that node. Because every pseudo-NMOS stage is inherently inverting, an extra inverter stage is used wherever the final, non-inverted polarity of a product term or of the output is required. This series/parallel NMOS approach is used throughout the ROM, PLA and PAL transistor-level designs in this project.

13. Testbench and Simulation Programs

A testbench is a Verilog module used to apply input signals to the circuit under test and observe its outputs. All six designs below were simulated for all 16 possible combinations of A, B, C and D in EDA Playground, with VCD waveform generation enabled for EPWave analysis.

13.1 ROM - Gate Level

Design code:

```
`timescale 1ns/1ps

module ROM_Gate_Level(
    input A, B, C, D,
    output Y1
);

    wire nA, nB, nC, nD;
    wire P1, P2, P3;

    not(nA, A);
    not(nB, B);
    not(nC, C);
    not(nD, D);

    // Y1 = A'C'D + B'C'D + A'B'D'
    and(P1, nA, nC, D);       // A'C'D
    and(P2, nB, nC, D);       // B'C'D
    and(P3, nA, nB, nD);      // A'B'D'

    or(Y1, P1, P2, P3);

endmodule
```

Testbench code:

```
`timescale 1ns/1ps

module tb_rom_gate;
    reg A, B, C, D;
    wire Y1;

    ROM_Gate_Level dut(.A(A), .B(B), .C(C), .D(D), .Y1(Y1));

    integer i;
    initial begin
        $dumpfile("rom_gate.vcd");
        $dumpvars(0, tb_rom_gate);
        $display("A B C D | Y1");
        for (i = 0; i < 16; i = i + 1) begin
            {A,B,C,D} = i;
            #10;
            $display("%b %b %b %b |  %b", A,B,C,D,Y1);
        end
        $finish;
    end
endmodule
```

13.2 ROM - Pseudo-NMOS Level

Design code:

```
`timescale 1ns/1ps

module ROM_NMOS(
    input A, B, C, D,
    output Y1
);
    supply0 GND;
    wire nA, nB, nC, nD;

    pullup(nA); nmos(nA, GND, A);
    pullup(nB); nmos(nB, GND, B);
    pullup(nC); nmos(nC, GND, C);
    pullup(nD); nmos(nD, GND, D);

    wire P1, P2, P3;

    // P1 = A'C'D
    pullup(P1);
    nmos(P1, GND, A);
    nmos(P1, GND, C);
    nmos(P1, GND, nD);

    // P2 = B'C'D
    pullup(P2);
    nmos(P2, GND, B);
    nmos(P2, GND, C);
    nmos(P2, GND, nD);

    // P3 = A'B'D'
    pullup(P3);
    nmos(P3, GND, A);
    nmos(P3, GND, B);
    nmos(P3, GND, D);

    wire Y1_bar;
    pullup(Y1_bar);
    nmos(Y1_bar, GND, P1);
    nmos(Y1_bar, GND, P2);
    nmos(Y1_bar, GND, P3);

    pullup(Y1);
    nmos(Y1, GND, Y1_bar);

endmodule
```

Testbench code:

```
`timescale 1ns/1ps

module tb_rom_nmos;
    reg A, B, C, D;
    wire Y1;

    ROM_NMOS dut(.A(A), .B(B), .C(C), .D(D), .Y1(Y1));

    integer i;
    initial begin
        $dumpfile("rom_nmos.vcd");
        $dumpvars(0, tb_rom_nmos);
        $display("ROM - NMOS");
        $display("A B C D | Y1");
        for (i = 0; i < 16; i = i + 1) begin
```

```
            {A,B,C,D} = i;
            #10;
            $display("%b %b %b %b |  %b", A,B,C,D,Y1);
        end
        $finish;
    end
endmodule
```

13.3 PLA - Gate Level

Design code:

```
`timescale 1ns/1ps

module PLA_Gate_Level(
    input A, B, C, D,
    output Y1, Y2
);
    wire nA, nB, nC, nD;
    wire P1, P2, P3, P4;

    not(nA, A);
    not(nB, B);
    not(nC, C);
    not(nD, D);

    and(P1, nA, nC, D);   // P1 = A'C'D   (shared)
    and(P2, nA, nB, nD);  // P2 = A'B'D'  (shared)
    and(P3, nB, nC, D);   // P3 = B'C'D   (Y1 only)
    and(P4, nB, nC);      // P4 = B'C'    (Y2 only)

    or(Y1, P1, P2, P3);   // Y1 = A'C'D + A'B'D' + B'C'D
    or(Y2, P1, P2, P4);   // Y2 = A'C'D + A'B'D' + B'C'

endmodule
```

Testbench code:

```
`timescale 1ns/1ps

module tb_pla_gate;
    reg A, B, C, D;
    wire Y1, Y2;

    PLA_Gate_Level dut(.A(A), .B(B), .C(C), .D(D), .Y1(Y1), .Y2(Y2));

    integer i;
    initial begin
        $dumpfile("pla_gate.vcd");
        $dumpvars(0, tb_pla_gate);
        $display("A B C D | Y1 Y2");
        for (i = 0; i < 16; i = i + 1) begin
            {A,B,C,D} = i;
            #10;
            $display("%b %b %b %b |  %b  %b", A,B,C,D,Y1,Y2);
        end
        $finish;
    end
endmodule
```

13.4 PLA - Pseudo-NMOS Level

Design code:

```
`timescale 1ns/1ps

module PLA_NMOS(
    input A, B, C, D,
    output Y1, Y2
);
    supply0 GND;
    wire nA, nB, nC, nD;

    pullup(nA); nmos(nA, GND, A);
    pullup(nB); nmos(nB, GND, B);
    pullup(nC); nmos(nC, GND, C);
    pullup(nD); nmos(nD, GND, D);

    wire P1, P2, P3, P4;

    // P1 = A'C'D
    pullup(P1); nmos(P1,GND,A); nmos(P1,GND,C); nmos(P1,GND,nD);

    // P2 = A'B'D'
    pullup(P2); nmos(P2,GND,A); nmos(P2,GND,B); nmos(P2,GND,D);

    // P3 = B'C'D
    pullup(P3); nmos(P3,GND,B); nmos(P3,GND,C); nmos(P3,GND,nD);

    // P4 = B'C'
    pullup(P4); nmos(P4,GND,B); nmos(P4,GND,C);

    // Y1 = P1 + P2 + P3
    wire Y1_bar;
    pullup(Y1_bar);
    nmos(Y1_bar,GND,P1); nmos(Y1_bar,GND,P2); nmos(Y1_bar,GND,P3);
    pullup(Y1); nmos(Y1,GND,Y1_bar);

    // Y2 = P1 + P2 + P4
    wire Y2_bar;
    pullup(Y2_bar);
    nmos(Y2_bar,GND,P1); nmos(Y2_bar,GND,P2); nmos(Y2_bar,GND,P4);
    pullup(Y2); nmos(Y2,GND,Y2_bar);

endmodule
```

Testbench code:

```
`timescale 1ns/1ps

module tb_pla_nmos;
    reg A, B, C, D;
    wire Y1, Y2;

    PLA_NMOS dut(.A(A), .B(B), .C(C), .D(D), .Y1(Y1), .Y2(Y2));

    integer i;
    initial begin
        $dumpfile("pla_nmos.vcd");
        $dumpvars(0, tb_pla_nmos);
        $display("PLA - NMOS");
        $display("A B C D | Y1 Y2");
        for (i = 0; i < 16; i = i + 1) begin
            {A,B,C,D} = i;
            #10;
            $display("%b %b %b %b |  %b  %b", A,B,C,D,Y1,Y2);
```

```
        end
        $finish;
    end
endmodule
```

13.5 PAL - Gate Level

Design code:

```
`timescale 1ns/1ps

module PAL_Gate_Level(
    input A, B, C, D,
    output Y1, Y2
);
    wire nA, nB, nC, nD;
    wire P1, P2, P3;
    wire Q1, Q2, Q3;

    not(nA, A);
    not(nB, B);
    not(nC, C);
    not(nD, D);

    // Y1 product terms
    and(P1, nA, nC, D);   // A'C'D
    and(P2, nB, nC, D);   // B'C'D
    and(P3, nA, nB, nD);  // A'B'D'

    // Y2 product terms
    and(Q1, nB, nC);      // B'C'
    and(Q2, nA, nC, D);   // A'C'D
    and(Q3, nA, nB, nD);  // A'B'D'

    // Fixed OR plane
    or(Y1, P1, P2, P3);
    or(Y2, Q1, Q2, Q3);

endmodule
```

Testbench code:

```
`timescale 1ns/1ps

module tb_pal_gate;
    reg A, B, C, D;
    wire Y1, Y2;

    PAL_Gate_Level dut(.A(A), .B(B), .C(C), .D(D), .Y1(Y1), .Y2(Y2));

    integer i;
    initial begin
        $dumpfile("pal_gate.vcd");
        $dumpvars(0, tb_pal_gate);
        $display("A B C D | Y1 Y2");
        for (i = 0; i < 16; i = i + 1) begin
            {A,B,C,D} = i;
            #10;
            $display("%b %b %b %b |  %b  %b", A,B,C,D,Y1,Y2);
        end
        $finish;
    end
endmodule
```

13.6 PAL - Pseudo-NMOS Level

Design code:

```
`timescale 1ns/1ps

module PAL_NMOS(
    input A, B, C, D,
    output Y1, Y2
);
    supply0 GND;
    wire nA, nB, nC, nD;

    pullup(nA); nmos(nA, GND, A);
    pullup(nB); nmos(nB, GND, B);
    pullup(nC); nmos(nC, GND, C);
    pullup(nD); nmos(nD, GND, D);

    wire P1, P2, P3;
    // P1 = A'C'D
    pullup(P1); nmos(P1,GND,A); nmos(P1,GND,C); nmos(P1,GND,nD);
    // P2 = B'C'D
    pullup(P2); nmos(P2,GND,B); nmos(P2,GND,C); nmos(P2,GND,nD);
    // P3 = A'B'D'
    pullup(P3); nmos(P3,GND,A); nmos(P3,GND,B); nmos(P3,GND,D);

    wire Q1, Q2, Q3;
    // Q1 = B'C'
    pullup(Q1); nmos(Q1,GND,B); nmos(Q1,GND,C);
    // Q2 = A'C'D
    pullup(Q2); nmos(Q2,GND,A); nmos(Q2,GND,C); nmos(Q2,GND,nD);
    // Q3 = A'B'D'
    pullup(Q3); nmos(Q3,GND,A); nmos(Q3,GND,B); nmos(Q3,GND,D);

    // Fixed OR plane for Y1
    wire Y1_bar;
    pullup(Y1_bar);
    nmos(Y1_bar,GND,P1); nmos(Y1_bar,GND,P2); nmos(Y1_bar,GND,P3);
    pullup(Y1); nmos(Y1,GND,Y1_bar);

    // Fixed OR plane for Y2
    wire Y2_bar;
    pullup(Y2_bar);
    nmos(Y2_bar,GND,Q1); nmos(Y2_bar,GND,Q2); nmos(Y2_bar,GND,Q3);
    pullup(Y2); nmos(Y2,GND,Y2_bar);

endmodule
```

Testbench code:

```
`timescale 1ns/1ps

module tb_pal_nmos;
    reg A, B, C, D;
    wire Y1, Y2;

    PAL_NMOS dut(.A(A), .B(B), .C(C), .D(D), .Y1(Y1), .Y2(Y2));

    integer i;
    initial begin
        $dumpfile("pal_nmos.vcd");
        $dumpvars(0, tb_pal_nmos);
        $display("PAL - NMOS");
        $display("A B C D | Y1 Y2");
        for (i = 0; i < 16; i = i + 1) begin
            {A,B,C,D} = i;
```

```
            #10;
            $display("%b %b %b %b |  %b  %b", A,B,C,D,Y1,Y2);
        end
        $finish;
    end
endmodule
```

14. Result Analysis

The simulation verifies the Boolean functions for all 16 possible combinations of A, B, C, D. The expected output behaviour, common to the ROM (Y1 only), PLA and PAL designs, both at gate level and pseudo-NMOS level, is shown below.

A	B	C	D	Y1	Y2
0	0	0	0	1	1
0	0	0	1	1	1
0	0	1	0	1	1
0	0	1	1	0	0
0	1	0	0	0	0
0	1	0	1	1	1
0	1	1	0	0	0
0	1	1	1	0	0
1	0	0	0	0	1
1	0	0	1	1	1
1	0	1	0	0	0
1	0	1	1	0	0
1	1	0	0	0	0
1	1	0	1	0	0
1	1	1	0	0	0
1	1	1	1	0	0

All six implementations (ROM gate/NMOS, PLA gate/NMOS, PAL gate/NMOS) produced this same truth table when simulated in EDA Playground, confirming that the gate-level and pseudo-NMOS-level descriptions are functionally equivalent. The most notable row is input ABCD = 1000 (decimal 8), where Y1 = 0 but Y2 = 1 — this single difference is the direct consequence of digit 8 appearing only in the preceding roll number 2109058 and not in the student's own roll number 2109059.

15. Discussion

This project provides a practical understanding of programmable logic structures and their implementation using Verilog HDL.

First, the ROM implementation demonstrates how a Boolean function can be represented by selecting particular minterms. The ROM generates only the required output Y1, and the selected minterms (0, 1, 2, 5, 9) are combined through an OR operation, using the reduced form Y1 = A'C'D + B'C'D + A'B'D' at the gate level.

Second, the PLA implementation demonstrates the use of common product terms. The terms P1 = A'C'D and P2 = A'B'D' are shared between Y1 and Y2, while P3 = B'C'D is required only for Y1 and P4 = B'C' only for Y2. This makes it possible to share hardware between the two outputs, reducing the number of physically generated AND-plane terms from six per-minterm terms to just four shared/unique terms.

Third, the PAL implementation uses the same required product terms and output equations, but represents the PAL structure, where the OR connections are treated as fixed rather than programmable, and where the shared terms A'C'D and A'B'D' must be regenerated separately for each output's own AND-plane row.

The project also demonstrates the difference between gate-level modelling and pseudo-NMOS-level modelling. Gate-level Verilog describes the circuit using logic primitives such as not, and, and or, while pseudo-NMOS-level Verilog goes one step deeper and describes the logic using pull-up loads combined with series/parallel NMOS pull-down networks.

Another important part of the project is simulation. Testing all 16 possible input combinations ensures that each circuit does not only work for a few selected test cases. The testbenches used in this project systematically apply all 16 input combinations and display the corresponding outputs, and the waveform generated through the VCD file provides a visual way to verify the relationship between the input signals and the output signals.

16. Conclusion

The project successfully demonstrates the design and simulation of ROM, PLA, and PAL using both gate-level and pseudo-NMOS-level Verilog modelling.

The required Boolean functions were implemented using the minterms:

$$Y1 = \Sigma m(0, 1, 2, 5, 9) \rightarrow Y1 = A'C'D + B'C'D + A'B'D'$$

$$Y2 = \Sigma m(0, 1, 2, 5, 8, 9) \rightarrow Y2 = B'C' + A'C'D + A'B'D'$$

The ROM implementation showed how selected minterms can be used to generate a single output function. The PLA implementation demonstrated the generation and sharing of product terms between multiple outputs, reducing the AND-plane hardware compared to a per-minterm approach. The PAL implementation demonstrated a similar product-term approach with the characteristic fixed OR-plane concept.

At the transistor level, the project further showed how basic digital gates can be constructed using pseudo-NMOS logic, with pull-up loads and series/parallel NMOS pull-down networks used as building blocks for the inverter, AND-like, and OR-like structures used throughout the larger circuits.

Finally, all possible 4-bit input combinations were tested through Verilog testbenches, and VCD waveform generation was included for EPWave analysis. Therefore, the project provides a complete practical understanding of Boolean logic, programmable logic architectures, Verilog HDL, transistor-level design, and digital circuit simulation.